

Narzędzia Wspierające Programowanie

Symulacja Jam2: zadanie do samodzielnej pracy

Jam2 jest symulatorem zderzeń jąder atomowych. Twoim celem jest napisanie skryptów podających seryjnie na farmę kilka zadań z procesem Jam2, tak aby nie zmieszać plików wyjściowych i logów.

- ▶ Utwórz u siebie katalog „jam2”, wejdź do niego i skopiuj tam zestaw plików:

```
$ cp -pr /home/2/kpias/jam2/serialize .
```

jam : link symboliczny do aplikacji.

jam.inp : plik sterujący z fizyką (tu: żądamy 10 zderzeń ^{58}Ni z ^{58}Ni przy energii 4 GeV/nukl.)

- ▶ Umieść lokalizację Twojego folderu w tym miejscu: [[LINK do G.Docs](#)]

- ▶ Celem Twoich działań i skryptów jest zbudowanie takiej struktury na Twoim katalogu „jam2” :

./jam	j.w.; link będzie wzorcem do skopiowania przez cp -pr)
./jam.inp	j.w.; plik będzie wzorcem do skopiowania przez cp -p)
./{Twój Submission script}	puszcza zestaw zadań, jedno po drugim
./{Twój Task script}	dla danego zadania: ustawia ścieżki i włącza runqmd.bash
./logs	katalog do logów
./runs	katalog z podkatalogami dla kolejnych run'ów

+→ ./run1/ katalog dla runu 1

./jam {tylko link}

./jam.inp

{plik wyjściowy z symulacji: phase.dat.gz }

+→ ./run2/ katalog dla runu 2

./jam {tylko link}

./jam.inp

{plik wyjściowy z symulacji: phase.dat.gz }

+→ katalog dla runu

- ▶ Submission script: możesz skopiować jego wzorzec z wcześniejszych symulacji. Zmień zasoby per zadanie na: 10 minut, 1000 megabajtów. Zapewnij, aby skrypt tworzył katalog logs oraz katalog runs . Wariant puszczenia procesu jest wprawdzie zakomentowany, na rzecz testowania lokalnego. Wprawdzie sprawdź, czy wszystko działa lokalnie. Dopiero później podaj zadanie na farmę.

(c.d. na następnej stronie)

- ▶ Skrypt poświęcony jednemu zadaniu możesz nazwać np. `jobScript.sh` . Powinien mieć w sobie:
 - Powinien mieć przekazaną z Submission Scriptu zmienną z numerem runu (np. o nazwie `run`).
 - Powinien wypisać powitanie, jaki kod zamierza puścić i jaki przyjął nr Runu (`echo ...`)
 - Z przyczyn technicznych dodaj dwie zmienne środowiskowe:


```
export PYTHIA8=/home/2/kpias/soft/pythia8307/
export PYTHIA8DATA=/home/2/kpias/soft/pythia8307/share/Pythia8/xmldoc
```
 - Pamiętaj, na której ścieżce `jobScript` teraz operuje ...
 - utwórz zmienne z lokalizacjami plików “log” dla `stdout` i `stderr`, zawierającymi nr runu;
 - utwórz zmienną z lokalizacją foldera poświęconego danemu runowi, zawierającą jego nr. Skasuj lub wyczyść katalog nr runu (na wypadek, gdyby istniał wcześniej i/lub coś w nim było). Jeśli go skasowałeś/aś, to go utwórz od nowa.
 - Na ścieżce dot. nr runu, utwórz link symboliczny do kodu: `/home/2/kpias/soft/jam2/jam2/jam`
 - Tam też wkopiuj plik sterujący `jam.inp` .
 - przejdź do ścieżki dot. nr runu.
 - Włącz `jam`: `./jam` , przekierowując `stdout` i `stderr` do odpowiednich plików “log”
 - ale wcześniej oraz później wystaw stempel daty (`date`)
 - Nakaz rozpakowanie pliku wynikowego:
- ▶ Proponuję zapuścić `submission script` najpierw tylko lokalnie i tylko dla runu 1. Sprawdź, czy symulacja działa (`top`) . Po zakończeniu: Sprawdź poprawność utworzonych katalogów i skopiowanych plików. Sprawdź, czy symulacja wytworzyła plik w prawidłowym miejscu
- ▶ Zapuć ponownie `submission script` dla runu 1, aby sprawdzić, czy nadpisywanie działa.
- ▶ Teraz zapuć `submission script` dla runów 2 + 3. To prosty test serializacji. Przy okazji sprawdź, czy `jobScript` nie skasował katalogu dla runu 1 (nie powinien).
- ▶ Teraz w Submission Script zakomentuj włączanie lokalnej symulacji i uaktywnij podanie jej na farmę. Sprawdź działanie tylko dla runu 4. Śledź: status kolejki, pliki z logami, co się tworzy w `./runs/run4` .
- ▶ Teraz zapuć Submission Script dla runów 1 – 5 . To Twoja docelowa symulacja. Sprawdź, czy paczka zadań idzie? Czy wszystko jest ok? Skoryguj ew. problemy.

Powodzenia!